<table>
<tr><td></td><td colspan="2">Marwadi University
Faculty of Engineering &Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Digital Signal and Image Processing (01CT1513)</td><td colspan="2">AIM: Simulate discrete time sequences.</td></tr>
<tr><td>Experiment No: 1</td><td>Date:19/07/2026</td><td>Enrollment No:92400133132</td></tr>
</table>

To plot unit impulse signal using Python.

In signal processing, a unit impulse signal, also known as a Dirac delta function or impulse function, is a mathematical function that has the value of 1 at time zero and zero elsewhere. Generating a unit impulse signal in Python can be done using the NumPy library.

Steps:

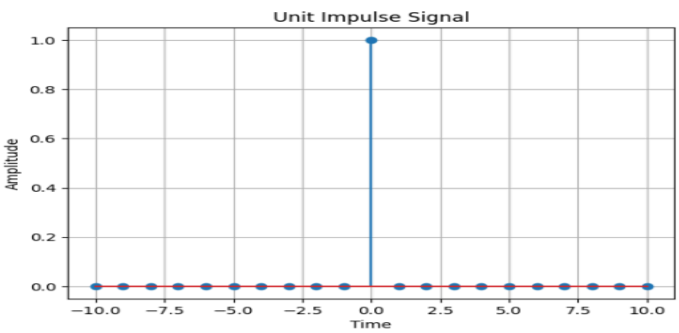
- First, we import the necessary libraries: numpy for numerical computations and matplotlib.pyplot for plotting.
- Next, we define a function called unit_impulse that takes two arguments: length (the length of the signal) and position (the position of the impulse within the signal). Inside the unit_impulse function, we create an array of zeros with the specified length using np.zeros(length).
- We set the value at the specified position to 1, representing the impulse.
- We define the parameters **start**, **stop**, and **step** to specify the x-axis range and step size.
- We use np.arange to generate the x-axis values. The np.arange function creates an array of numbers from start to stop (inclusive) with a step size of step.
- We calculate the length of the impulse signal using len(x), which represents the number of x-axis values.
- We modify the unit_impulse function call to calculate the position of the impulse based on the x-axis range. Since we want the impulse at n=0, we set the position to abs(start)//step, which calculates the index of 0 in the x-axis array.
- abs(start): The abs() function returns the absolute value of the start variable. This is done to ensure that the result is always positive, regardless of whether start is positive or negative.
- abs(start)//step: The // operator performs integer division, which discards the decimal part of the division result and returns the integer quotient. In this case, abs(start)//step calculates the number of steps from the start value to reach the position of 0 in the x-axis array.
- The unit_impulse function returns the generated signal.
- By using abs(start)//step as the position argument in the unit_impulse function call, we ensure that the impulse is correctly positioned at n=0 within the generated unit impulse signal.
- For example, if start = -10 and step = 1, the expression abs(start)//step evaluates to (10)//(1), which equals 10. This means that the impulse will be placed at index 10 in the signal, aligning it with n=0.
- Using abs(start)//step allows the code to handle both positive and negative start values correctly, ensuring that the unit impulse is positioned accurately regardless of the specified x-axis range and step size.
- We then define the desired length and position for the unit impulse signal.
- We generate the unit impulse signal by calling the unit_impulse function with the specified parameters.
- Finally, we plot the unit impulse signal using plt.stem to create a stem plot, and then display the plot using plt.show().

- When you run this code, it will generate a stem plot showing the unit impulse signal with a spike at the specified position (in this case, position 10).
- plt.grid(True) in the code helps to improve the visual representation of the plot by adding gridlines, making it easier to analyze.
- Note: To run this code, you'll need to have the NumPy and Matplotlib libraries installed. You can install them using pip install numpy matplotlib.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
def unit_impulse(length, position):
  signal = np.zeros(length)
  signal[position] = 1
  return signal
# Parameters
start = -10 # Start value of the x-axis range
stop = 10 # Stop value of the x-axis range
step = 1 # Step size
# Generate x-axis values
x = np.arange(start, stop+step, step)
# Generate unit impulse signal
impulse_signal = unit_impulse(len(x), abs(start)//step)
# Plot the signal
plt.stem(x, impulse_signal)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Impulse Signal')
plt.grid(True)
plt.show()
```

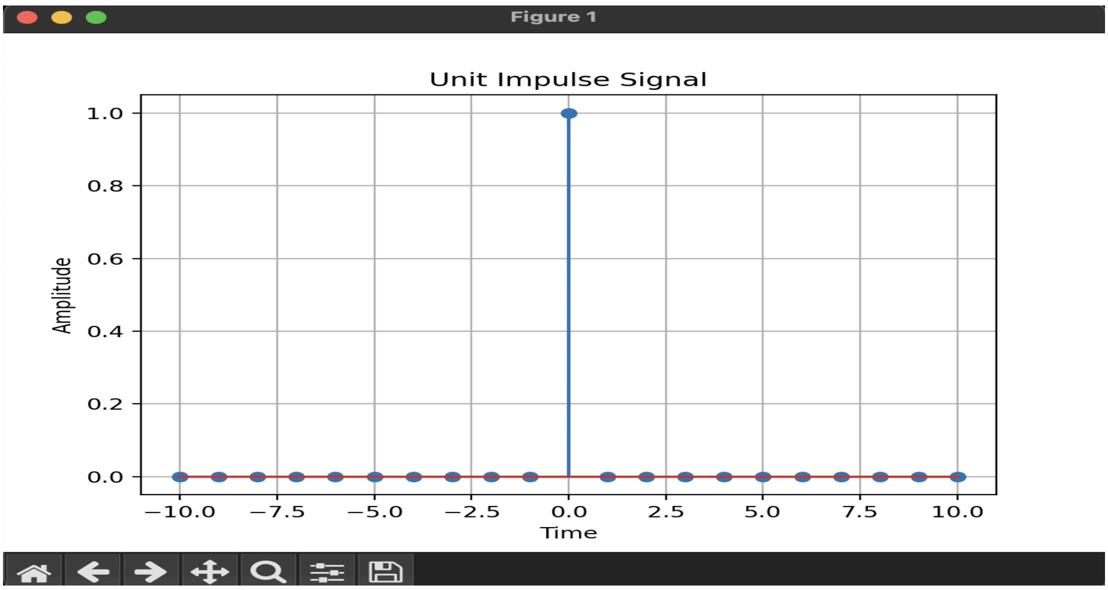
Output:



Marwadi University NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate discrete time sequences.**

Experiment No: 1	**Date:19/07/2026**	**Enrollment No:92400133132**

```
import numpy as np
import matplotlib.pyplot as plt

def unit_impulse(length,position):
   signal=np.zeros(length)
   signal[position]=1
   return signal

#parameters
start=-10
stop=10
step=1

x=np.arange(start,stop+step,step)
impulse_signal=unit_impulse(len(x),abs(start)//step)

plt.stem(x,impulse_signal)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Impulse Signal')
plt.grid(True)
plt.show()
```



<table>
<tr><td rowspan="1"></td><td colspan="2">Marwadi University
Faculty of Engineering &Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Digital Signal and Image Processing (01CT1513)</td><td colspan="2">AIM: Simulate discrete time sequences.</td></tr>
<tr><td>Experiment No: 1</td><td>Date:19/07/2026</td><td>Enrollment No:92400133132</td></tr>
</table>

AIM
Simulate impulse train
Theory:
An impulse train, also known as a Dirac comb, is a periodic sequence of impulses. Each impulse has an amplitude of 1 and occurs at regular intervals. The impulse train is often used in signal processing to model periodic phenomena or to analyze the frequency response of Systems.

The impulse train is defined as follows:
impulse_train(n, period) = 1, if n % period == 0
= 0, otherwise
where n is the sample index and period is the period of the impulse train.

Flowchart:
The flowchart for the program will consist of the following steps:

Define the parameters for the impulse train: signal length and period.
Create an empty array to store the impulse train.
Iterate over each sample index.
Check if the sample index is a multiple of the period. If yes, assign the value 1 to the impulse train array at that index; otherwise, assign the value 0.
Plot and display the impulse train.
Save the impulse train array (optional).
Now, let's see the Python program that simulates an impulse train:
Program
```
import numpy as np
import matplotlib.pyplot as plt
def simulate_impulse_train(signal_length, period):
  impulse_train = np.zeros(signal_length)
  for n in range(signal_length):
     if n % period == 0:
        impulse_train[n] = 1
  return impulse_train

# Define the parameters for the impulse train
signal_length = 100 # Length of the impulse train
period = 10 # Period of the impulse train

# Simulate the impulse train
impulse_train = simulate_impulse_train(signal_length, period)
```

```
# Plot and display the impulse train
plt.stem(impulse_train)
plt.title('Impulse Train')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.show()

# Save the impulse train array (optional)
# np.savetxt('impulse_train.txt', impulse_train, delimiter=',')
```

Explanation:
We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.
The simulate_impulse_train function takes the signal length and period as input and returns an array representing the impulse train.
In the main part of the program, we define the parameters for the impulse train: signal_length (the length of the impulse train) and period (the period of the impulse train).
We simulate the impulse train by calling the simulate_impulse_train function with the specified parameters.
We plot and display the impulse train using matplotlib.pyplot.stem.
The impulse train array can be optionally saved to a file using numpy.savetxt.
You can modify the signal_length and period variables to change the length and period of the impulse train, respectively.

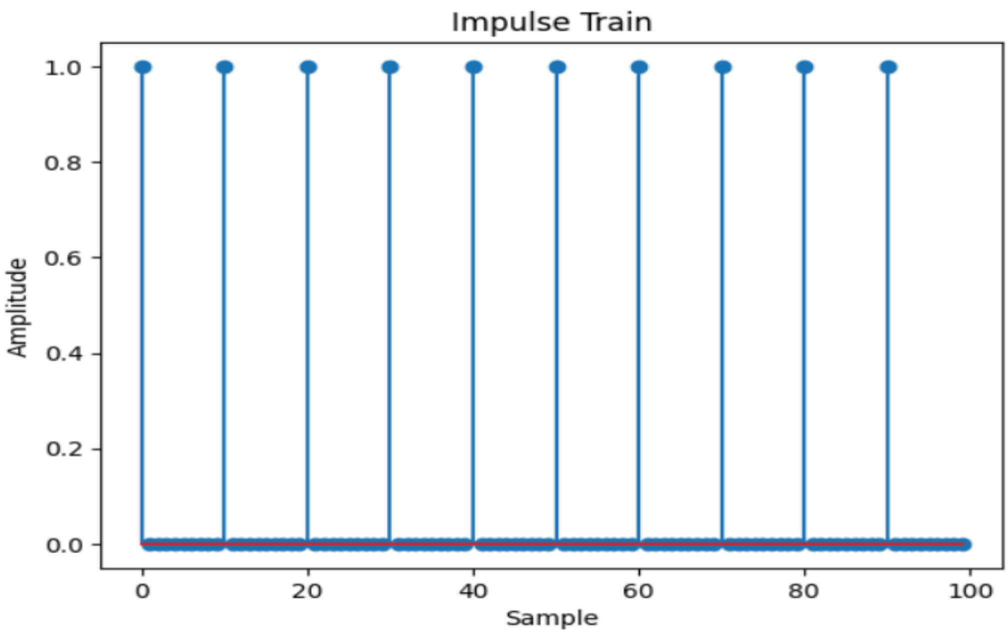
Uncomment the np.savetxt line if you want to save the impulse train array to a text file.

Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:

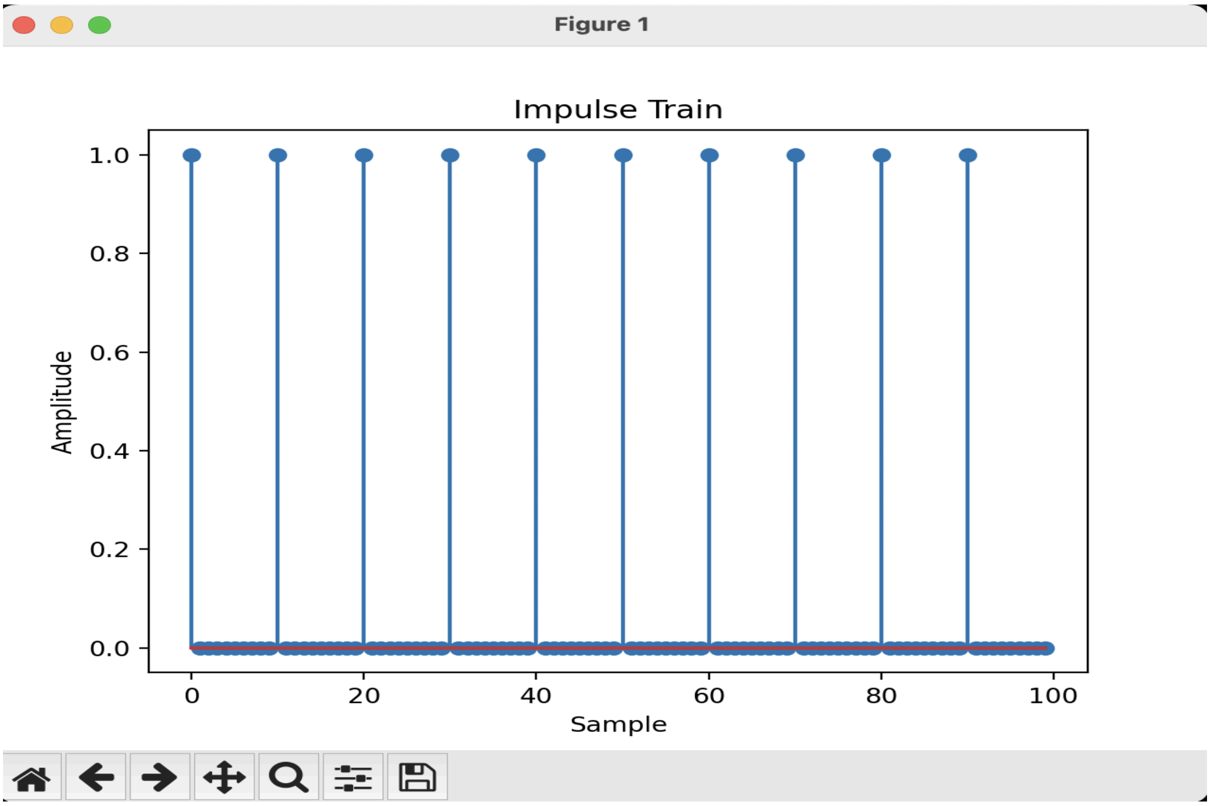
A stem plot will be displayed showing the impulse train.
The impulse train array will be saved to a text file (optional).
Feel free to experiment with different parameters for the impulse train and observe the output.

-




```
import numpy as np
import matplotlib.pyplot as plt

def simulate_unit_impulse_train(Signal_length,period):
    impulse_train=np.zeros(Signal_length)
    for n in range(Signal_length):
        if n%period==0:
            impulse_train[n]=1
    return impulse_train

Signal_length=100
period =10
impulse_train=simulate_unit_impulse_train(Signal_length,period)

plt.stem(impulse_train)
plt.title('Impulse Train')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.show()
```

Marwadi University (Marwadi Chandarana Group), NAAC A+	Marwadi University Faculty of Engineering &Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Simulate discrete time sequences.	
Experiment No: 1	Date:19/07/2026	Enrollment No:92400133132



AIM: Write a python program to Simulate continuous and discrete unit step signal,
Theory:
A unit step signal, also known as a Heaviside step function, is a signal that is 0 for negative time or indices and 1 for non-negative time or indices. It is commonly used to represent a sudden change or transition in a system.

The continuous unit step function is defined as:
u(t) = 0, t < 0
= 1, t >= 0
The discrete unit step function is defined as:
u[n] = 0, n < 0
= 1, n >= 0
Flowchart:
The flowchart for the program will consist of the following steps:
Define the time range (for the continuous signal) or the number of samples (for the discrete signal).
Create an empty array to store the unit step signal.
Iterate over each time or sample index.

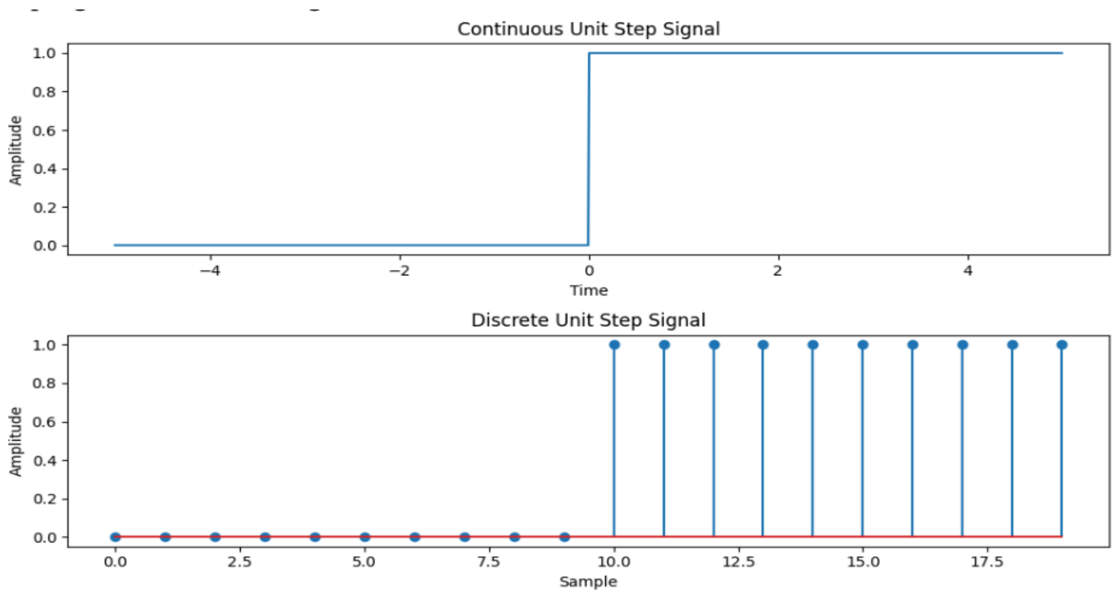
Set the value of the unit step signal to 0 for negative time or indices, and 1 for non-negative time or indices.
Plot and display the unit step signal.
Save the unit step signal array (optional).
Now, let's see the Python program that simulates continuous and discrete unit step signals:
Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_unit_step(time):
unit_step = np.zeros_like(time)
unit_step[time >= 0] = 1
return unit_step
def simulate_discrete_unit_step(num_samples):
unit_step = np.zeros(num_samples)
unit_step[num_samples // 2:] = 1
return unit_step
# Define the time range for the continuous unit step signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5
# Simulate the continuous unit step signal
continuous_unit_step = simulate_continuous_unit_step(time)
# Define the number of samples for the discrete unit step signal
num_samples = 20 # Number of samples
# Simulate the discrete unit step signal
discrete_unit_step = simulate_discrete_unit_step(num_samples)
# Plot and display the continuous and discrete unit step signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_unit_step)
plt.title('Continuous Unit Step Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_unit_step)
plt.title('Discrete Unit Step Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
# Save the unit step signal arrays (optional)
# np.savetxt('continuous_unit_step.txt', continuous_unit_step, delimiter=',')
# np.savetxt('discrete_unit_step.txt', discrete_unit_step, delimiter=',')
```

Marwadi University (Marwadi Chandarana Group, NAAC A+)	Marwadi University Faculty of Engineering &Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate discrete time sequences.**	
Experiment No: 1	**Date:19/07/2026**	**Enrollment No:92400133132**

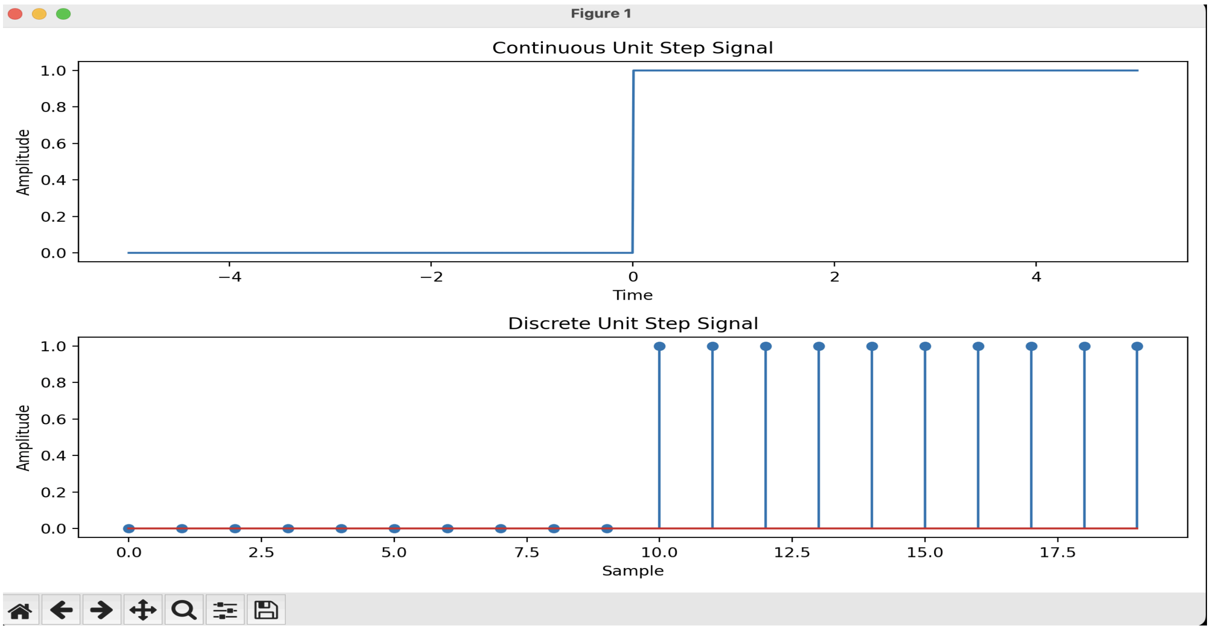
Explanation:
We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.
The simulate_continuous_unit_step function takes a time array as input and returns an array representing the continuous unit step signal. It creates an array of zeros with the same shape as the input time array and sets the values to 1 for non-negative time values.
The simulate_discrete_unit_step function takes the number of samples as input and returns an array representing the discrete unit step signal. It creates an array of zeros with the specified number of samples and sets the values to 1 starting from the middle index.
In the main part of the program, we define the time range for the continuous unit step signal using numpy.linspace.
We simulate the continuous unit step signal by calling the simulate_continuous_unit_step function with the time array.
We define the number of samples for the discrete unit step signal.
We simulate the discrete unit step signal by calling the simulate_discrete_unit_step function with the number of samples.
We plot and display the continuous and discrete unit step signals using matplotlib.pyplot.plot and matplotlib.pyplot.stem, respectively.
The unit step signal arrays can be optionally saved to text files using numpy.savetxt.
You can modify the time range for the continuous unit step signal and the num_samples for the discrete unit step signal according to your requirements.
Uncomment the np.savetxt lines if you want to save the unit step signal arrays to text files.
Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:
Two plots will be displayed: the continuous unit step signal and the discrete unit step signal.
The unit step signal arrays will be saved to text files (optional).
Feel free to experiment with different time ranges and numbers of samples to generate unit step signals of various lengths.




```python
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_unit_step(time):
    unit_step = np.zeros_like(time)
    unit_step[time >= 0] = 1
    return unit_step
def simulate_discrete_unit_step(num_samples):
    unit_step = np.zeros(num_samples)
    unit_step[num_samples // 2:] = 1
    return unit_step

time = np.linspace(-5, 5, 1000)
continuous_unit_step = simulate_continuous_unit_step(time)

num_samples = 20

discrete_unit_step = simulate_discrete_unit_step(num_samples)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_unit_step)
```

```
plt.title('Continuous Unit Step Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_unit_step)
plt.title('Discrete Unit Step Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



AIM: Write a python program to Simulate continuous and discrete ramp signal,

Theory:

A ramp signal is a signal that varies linearly with time. It starts at an initial value and increases or decreases at a constant rate. Ramp signals are commonly used in various applications, such as signal processing, control systems, and digital communications.

The continuous ramp signal is defined as:

ramp(t) = a * t, for t >= 0

= 0, for t < 0

where a is the slope of the ramp.

The discrete ramp signal is defined as:

ramp[n] = a * n, for n >= 0

= 0, for n < 0

where a is the slope of the ramp.

Flowchart:
The flowchart for the program will consist of the following steps:
Define the time range (for the continuous signal) or the number of samples (for the discrete signal).
Create an empty array to store the ramp signal.

Iterate over each time or sample index.
Set the value of the ramp signal to a * t for continuous or a * n for discrete, where a is the slope and t or n is the time or sample index.
Plot and display the ramp signal.
Save the ramp signal array (optional).
Now, let's see the Python program that simulates continuous and discrete ramp signals:
Program

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_ramp(time, slope):
ramp = np.zeros_like(time)
ramp[time >= 0] = slope * time[time >= 0]
return ramp
def simulate_discrete_ramp(num_samples, slope):
ramp = np.zeros(num_samples)
ramp[num_samples // 2:] = slope * np.arange(num_samples // 2, num_samples)
return ramp
# Define the time range for the continuous ramp signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5
# Define the number of samples and slope for the discrete ramp signal
num_samples = 20 # Number of samples
slope = 2 # Slope of the ramp
# Simulate the continuous ramp signal
continuous_ramp = simulate_continuous_ramp(time, slope)
# Simulate the discrete ramp signal
discrete_ramp = simulate_discrete_ramp(num_samples, slope)
# Plot and display the continuous and discrete ramp signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_ramp)
plt.title('Continuous Ramp Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_ramp)
plt.title('Discrete Ramp Signal')
```

```
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

# Save the ramp signal arrays (optional)
# np.savetxt('continuous_ramp.txt', continuous_ramp, delimiter=',')
# np.savetxt('discrete_ramp.txt', discrete_ramp, delimiter=',')
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.

The simulate_continuous_ramp function takes a time array and a slope as input and returns an array representing the continuous ramp signal. It creates an array of zeros with the same shape as the input time array and sets the values to slope * time for non-negative time values.

The simulate_discrete_ramp function takes the number of samples and a slope as input and returns an array representing the discrete ramp signal. It creates an array of zeros with the specified number of samples and sets the values to slope * n starting from the middle index.

In the main part of the program, we define the time range for the continuous ramp signal using numpy.linspace.

We define the number of samples and slope for the discrete ramp signal.

We simulate the continuous ramp signal by calling the simulate_continuous_ramp function with the time array and slope.

We simulate the discrete ramp signal by calling the simulate_discrete_ramp function with the number of samples and slope.

We plot and display the continuous and discrete ramp signals using matplotlib.pyplot.plot and matplotlib.pyplot.stem, respectively.

The ramp signal arrays can be optionally saved to text files using numpy.savetxt.

You can modify the time range for the continuous ramp signal, the num_samples for the discrete ramp signal, and the slope of the ramps according to your requirements.

Uncomment the np.savetxt lines if you want to save the ramp signal arrays to text files.

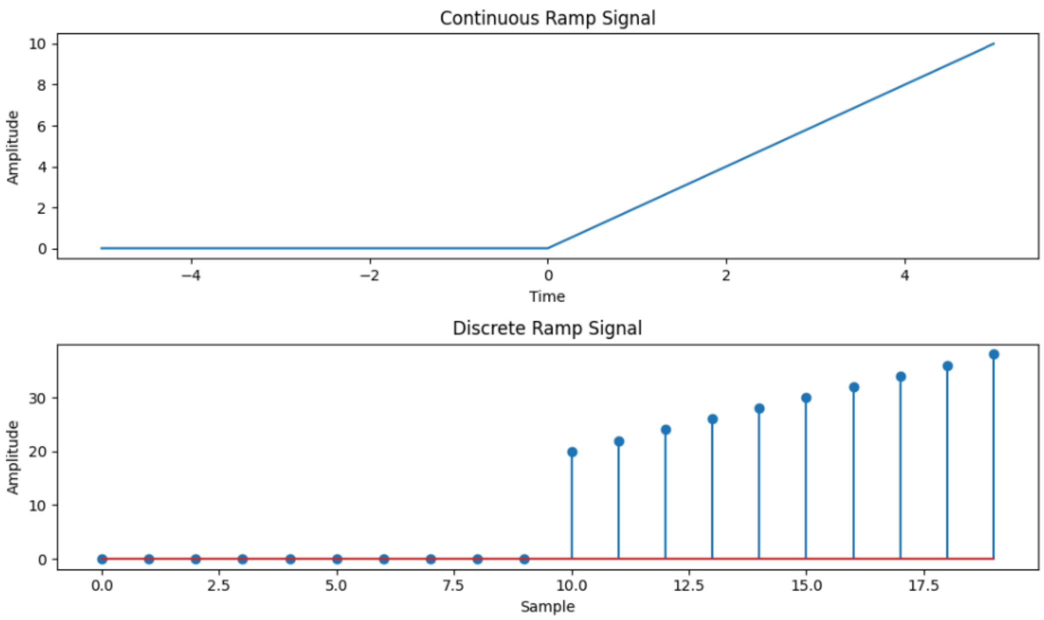
Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:

Two plots will be displayed: the continuous ramp signal and the discrete ramp signal.

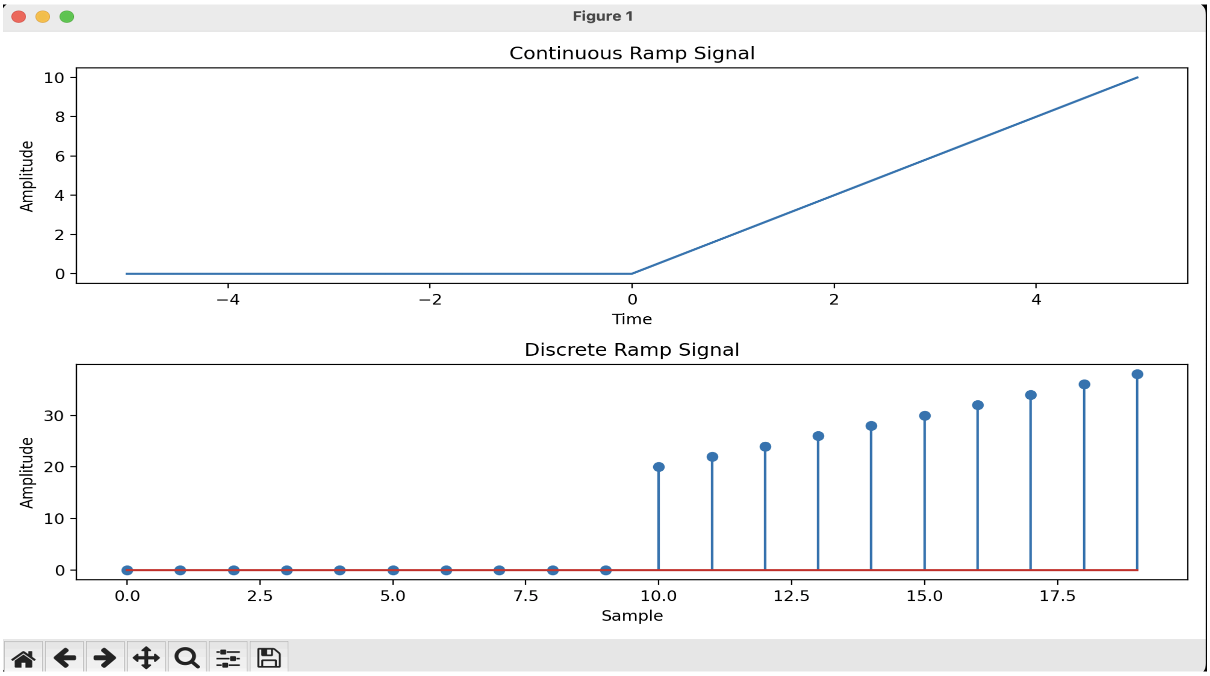
The ramp signal arrays will be saved to text files (optional).

Feel free to experiment with different time ranges, numbers of samples, and slopes to generate ramp signals of various lengths and slopes.




```python
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_ramp(time, slope):
    ramp = np.zeros_like(time)
    ramp[time >= 0] = slope * time[time >= 0]
    return ramp
def simulate_discrete_ramp(num_samples, slope):
    ramp = np.zeros(num_samples)
    ramp[num_samples // 2:] = slope * np.arange(num_samples // 2, num_samples)
    return ramp

time = np.linspace(-5, 5, 1000)

num_samples = 20
slope = 2
continuous_ramp = simulate_continuous_ramp(time, slope)

discrete_ramp = simulate_discrete_ramp(num_samples, slope)

plt.figure(figsize=(10, 6))
```

```
plt.subplot(2, 1, 1)
plt.plot(time, continuous_ramp)
plt.title('Continuous Ramp Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_ramp)
plt.title('Discrete Ramp Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



AIM: Write a python program to Simulate continuous and discrete exponential signal,

Theory:

An exponential signal is a signal whose amplitude changes exponentially with time or index. Exponential signals can exhibit either growth or decay behavior, depending on the sign of the exponential term.

The continuous exponential signal is defined as:

x(t) = A * exp(b * t)

where A is the initial amplitude, b is the exponential coefficient, and t is the time.

The discrete exponential signal is defined as:
x[n] = A * exp(b * n)
where A is the initial amplitude, b is the exponential coefficient, and n is the sample index.

Flowchart:

The flowchart for the program will consist of the following steps:
Define the time range (for the continuous signal) or the number of samples (for the discrete signal).
Create an empty array to store the exponential signal.
Iterate over each time or sample index.
Set the value of the exponential signal to A * exp(b * t) for continuous or A * exp(b * n) for discrete, where A is the initial amplitude, b is the exponential coefficient, and t or n is the time or sample index.
Plot and display the exponential signal.
Save the exponential signal array (optional).
Now, let's see the Python program that simulates continuous and discrete exponential signals:

Program

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_exponential(time, amplitude, coefficient):
exponential_signal = amplitude * np.exp(coefficient * time)
return exponential_signal
def simulate_discrete_exponential(num_samples, amplitude, coefficient):
exponential_signal = amplitude * np.exp(coefficient * np.arange(num_samples))
return exponential_signal
# Define the time range for the continuous exponential signal
time = np.linspace(0, 5, 1000) # Time range from 0 to 5
# Define the number of samples, initial amplitude, and coefficient for the discrete exponential
signal
num_samples = 20 # Number of samples
amplitude = 2 # Initial amplitude
coefficient = -0.5 # Exponential coefficient
# Simulate the continuous exponential signal
continuous_exponential = simulate_continuous_exponential(time, amplitude, coefficient)
# Simulate the discrete exponential signal
discrete_exponential = simulate_discrete_exponential(num_samples, amplitude, coefficient)
# Plot and display the continuous and discrete exponential signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_exponential)
plt.title('Continuous Exponential Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
```

```
plt.subplot(2, 1, 2)
plt.stem(discrete_exponential)
plt.title('Discrete Exponential Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
# Save the exponential signal arrays (optional)
# np.savetxt('continuous_exponential.txt', continuous_exponential, delimiter=',')
# np.savetxt('discrete_exponential.txt', discrete_exponential, delimiter=',')
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.

The simulate_continuous_exponential function takes a time array, initial amplitude, and exponential coefficient as input and returns an array representing the continuous exponential signal. It calculates the exponential signal using numpy.exp with the specified amplitude and coefficient.

The simulate_discrete_exponential function takes the number of samples, initial amplitude, and exponential coefficient as input and returns an array representing the discrete exponential signal. It calculates the exponential signal using numpy.exp with the specified amplitude and coefficient, applied element-wise to the range of sample indices.

In the main part of the program, we define the time range for the continuous exponential signal using numpy.linspace.

We define the number of samples, initial amplitude, and exponential coefficient for the discrete exponential signal.

We simulate the continuous exponential signal by calling the simulate_continuous_exponential function with the time array, amplitude, and coefficient.

We simulate the discrete exponential signal by calling the simulate_discrete_exponential function with the number of samples, amplitude, and coefficient.

We plot and display the continuous and discrete exponential signals using matplotlib.pyplot.plot and matplotlib.pyplot.stem, respectively.

The exponential signal arrays can be optionally saved to text files using numpy.savetxt.

You can modify the time range for the continuous exponential signal, the number of samples, initial amplitude, and exponential coefficient for the discrete exponential signal according to your requirements.

Uncomment the np.savetxt lines if you want to save the exponential signal arrays to text files.

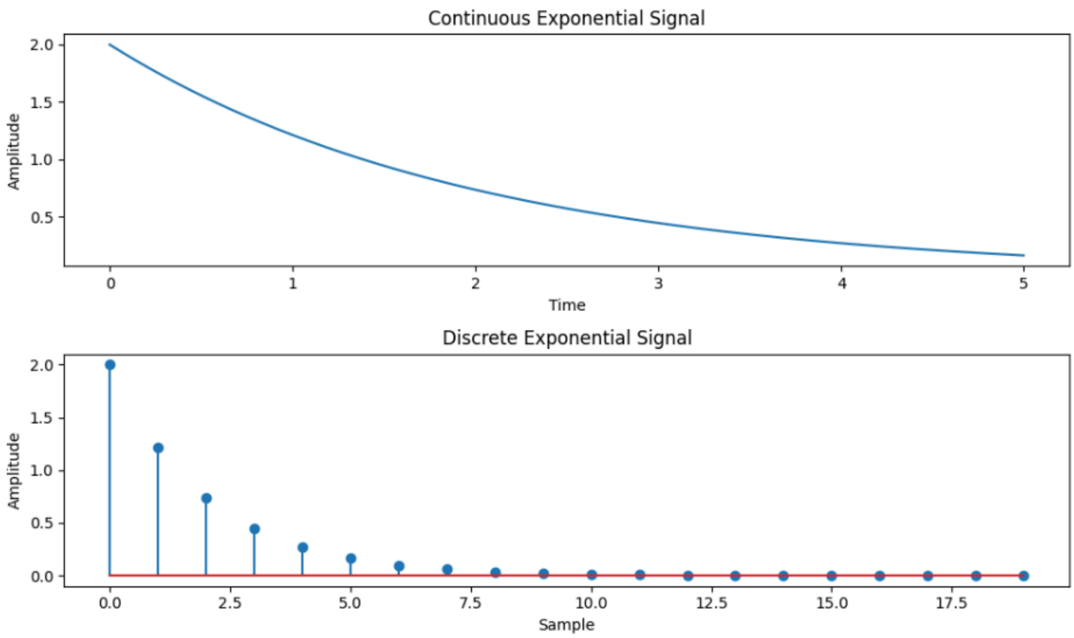
Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:

Two plots will be displayed: the continuous exponential signal and the discrete exponential signal.

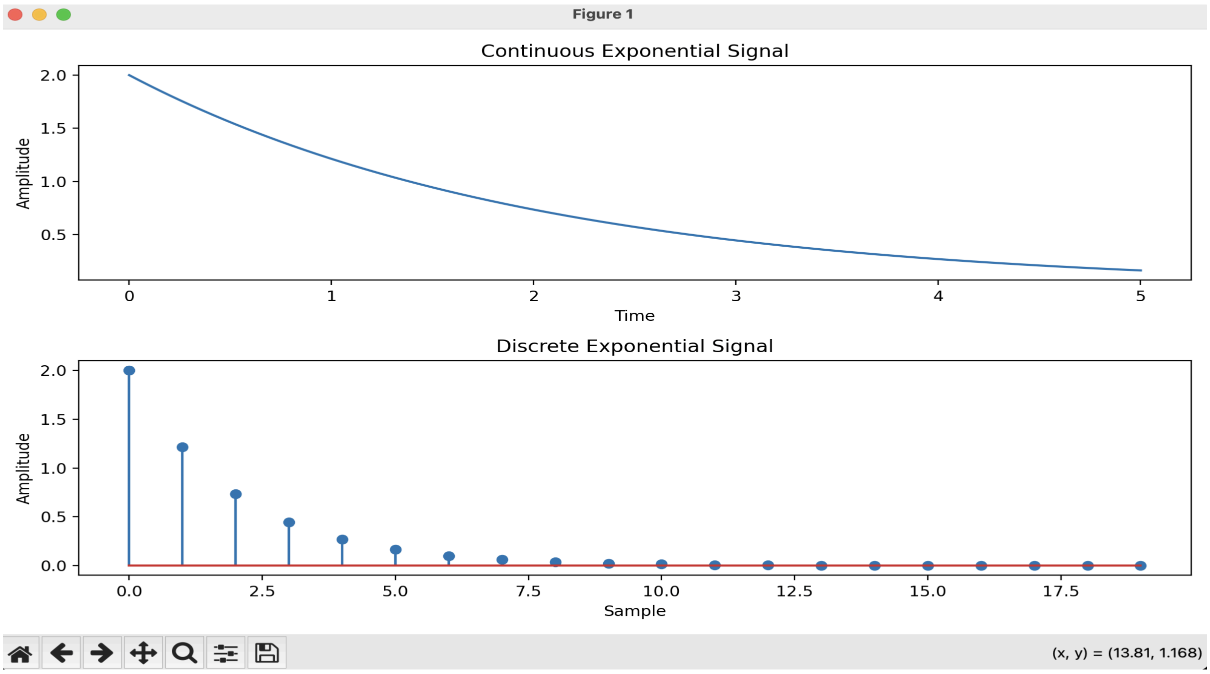
The exponential signal arrays will be saved to text files (optional).

Feel free to experiment with different time ranges, numbers of samples, initial amplitudes, and exponential coefficients to generate exponential signals of various shapes and characteristics.




```
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_exponential(time, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * time)
    return exponential_signal
def simulate_discrete_exponential(num_samples, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * np.arange(num_samples))
    return exponential_signal

time = np.linspace(0, 5, 1000)

num_samples = 20
amplitude = 2
coefficient = -0.5

continuous_exponential = simulate_continuous_exponential(time, amplitude,
coefficient)
```

```
discrete_exponential = simulate_discrete_exponential(num_samples, amplitude,
coefficient)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_exponential)
plt.title('Continuous Exponential Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_exponential)
plt.title('Discrete Exponential Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



Marwadi University, Marwadi Chandarana Group, NAAC A+	**Marwadi University** **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM: Simulate discrete time sequences.**	
Experiment No: 1	**Date:19/07/2026**	**Enrollment No:92400133132**

AIM: Write a python program to Simulate continuous and discrete parabolic signal
Theory:
A parabolic signal is a signal whose amplitude changes quadratically with time or index. It follows a parabolic curve and is commonly used to model various phenomena, such as projectile motion, signal processing, and data fitting.
The continuous parabolic signal is defined as:
parabolic(t) = a * t^2 + b * t + c
where a, b, and c are coefficients and t is the time.
The discrete parabolic signal is defined as:
parabolic[n] = a * n^2 + b * n + c
where a, b, and c are coefficients and n is the sample index.
Flowchart:
The flowchart for the program will consist of the following steps:
Define the time range (for the continuous signal) or the number of samples (for the discrete signal).
Create an empty array to store the parabolic signal.
Iterate over each time or sample index.
Set the value of the parabolic signal to a * t^2 + b * t + c for continuous or a * n^2 + b * n + c for discrete, where a, b, and c are the coefficients and t or n is the time or sample index.
Plot and display the parabolic signal.
Save the parabolic signal array (optional).
Now, let's see the Python program that simulates continuous and discrete parabolic signals:

Program:
```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_parabolic(time, coefficients):
parabolic_signal = np.polyval(coefficients, time)
return parabolic_signal
def simulate_discrete_parabolic(num_samples, coefficients):
parabolic_signal = np.polyval(coefficients, np.arange(num_samples))
return parabolic_signal
# Define the time range for the continuous parabolic signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5
# Define the number of samples and coefficients for the discrete parabolic signal
num_samples = 20 # Number of samples
coefficients = [1, 2, 1] # Coefficients of the parabolic signal
# Simulate the continuous parabolic signal
continuous_parabolic = simulate_continuous_parabolic(time, coefficients)
# Simulate the discrete parabolic signal
discrete_parabolic = simulate_discrete_parabolic(num_samples, coefficients)
# Plot and display the continuous and discrete parabolic signals
```

```
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_parabolic)
plt.title('Continuous Parabolic Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_parabolic)
plt.title('Discrete Parabolic Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
# Save the parabolic signal arrays (optional)
# np.savetxt('continuous_parabolic.txt', continuous_parabolic, delimiter=',')
# np.savetxt('discrete_parabolic.txt', discrete_parabolic, delimiter=',')
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.

The simulate_continuous_parabolic function takes a time array and a list of coefficients as input and returns an array representing the continuous parabolic signal. It calculates the parabolic signal using numpy.polyval with the specified coefficients and time array.

The simulate_discrete_parabolic function takes the number of samples and a list of coefficients as input and returns an array representing the discrete parabolic signal. It calculates the parabolic signal using numpy.polyval with the specified coefficients and a range of sample indices.

In the main part of the program, we define the time range for the continuous parabolic signal using numpy.linspace.

We define the number of samples and coefficients for the discrete parabolic signal.

We simulate the continuous parabolic signal by calling the simulate_continuous_parabolic function with the time array and coefficients.

We simulate the discrete parabolic signal by calling the simulate_discrete_parabolic function with the number of samples and coefficients.

We plot and display the continuous and discrete parabolic signals using matplotlib.pyplot.plot and matplotlib.pyplot.stem, respectively.

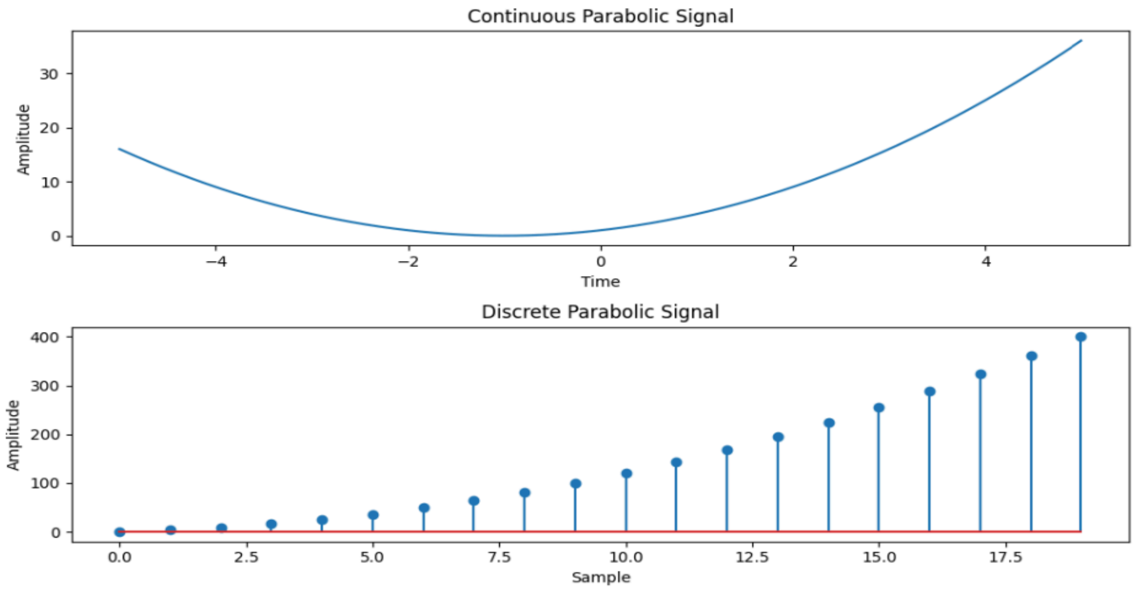
The parabolic signal arrays can be optionally saved to text files using numpy.savetxt.

You can modify the time range for the continuous parabolic signal, the number of samples, and the coefficients for the discrete parabolic signal according to your requirements.

Uncomment the np.savetxt lines if you want to save the parabolic signal arrays to text files.

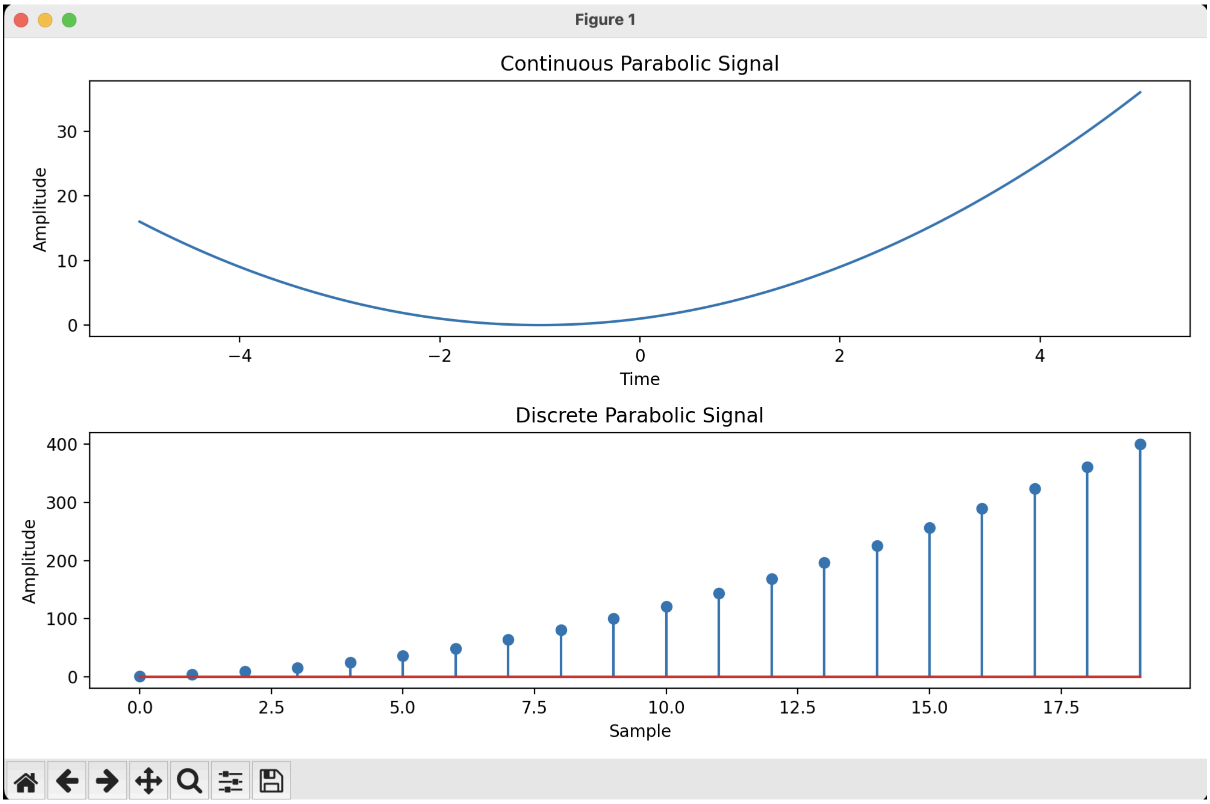
Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:
Two plots will be displayed: the continuous parabolic signal and the discrete parabolic signal.
The parabolic signal arrays will be saved to text files (optional).
Feel free to experiment with different time ranges, numbers of samples, and coefficients to generate parabolic signals of various shapes and characteristics.




```python
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_parabolic(time, coefficients):
    parabolic_signal = np.polyval(coefficients, time)
    return parabolic_signal
def simulate_discrete_parabolic(num_samples, coefficients):
    parabolic_signal = np.polyval(coefficients, np.arange(num_samples))
    return parabolic_signal

time = np.linspace(-5, 5, 1000)

num_samples = 20
coefficients = [1, 2, 1]

continuous_parabolic = simulate_continuous_parabolic(time, coefficients)

discrete_parabolic = simulate_discrete_parabolic(num_samples, coefficients)
```

```
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_parabolic)
plt.title('Continuous Parabolic Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_parabolic)
plt.title('Discrete Parabolic Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



AIM: Write a python program to Simulate continuous and discrete sine wave signal
Theory:
A sine wave signal is a continuous-time or discrete-time signal that follows a periodic oscillation described by the sine function. It is widely used in various applications, such as signal processing, communications, and audio synthesis.
The continuous sine wave signal is defined as:
sine(t) = A * sin(2 * pi * f * t + phi)
where A is the amplitude, f is the frequency, t is the time, and phi is the phase.
The discrete sine wave signal is defined as:
sine[n] = A * sin(2 * pi * f * n / Fs + phi)
where A is the amplitude, f is the frequency, n is the sample index, Fs is the sampling frequency, and phi is the phase.
Flowchart:
The flowchart for the program will consist of the following steps:
Define the time range (for the continuous signal) or the number of samples and the sampling frequency (for the discrete signal).
Create an empty array to store the sine wave signal.
Iterate over each time or sample index.
Set the value of the sine wave signal to A * sin(2 * pi * f * t + phi) for continuous or A * sin(2 * pi * f * n / Fs + phi) for discrete, where A, f, t, n, Fs, and phi are the corresponding parameters.
Plot and display the sine wave signal.
Save the sine wave signal array (optional).
Now, let's see the Python program that simulates continuous and discrete sine wave signals:
Program:

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_sine_wave(time, amplitude, frequency, phase):
sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
return sine_wave
def simulate_discrete_sine_wave(num_samples, sampling_frequency, amplitude, frequency,
phase):
time = np.arange(num_samples) / sampling_frequency
sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
return sine_wave
# Define the time range for the continuous sine wave signal
time = np.linspace(0, 1, 1000) # Time range from 0 to 1 second
# Define the number of samples, sampling frequency, and parameters for the discrete sine
wave signal
num_samples = 100 # Number of samples
sampling_frequency = 10 # Sampling frequency in Hz
amplitude = 1 # Amplitude of the sine wave
```

```
frequency = 2 # Frequency of the sine wave in Hz
phase = 0 # Phase angle of the sine wave in radians
# Simulate the continuous sine wave signal
continuous_sine_wave = simulate_continuous_sine_wave(time, amplitude, frequency, phase)
# Simulate the discrete sine wave signal
discrete_sine_wave = simulate_discrete_sine_wave(num_samples, sampling_frequency,
amplitude, frequency, phase)
# Plot and display the continuous and discrete sine wave signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_sine_wave)
plt.title('Continuous Sine Wave Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_sine_wave)
plt.title('Discrete Sine Wave Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
# Save the sine wave signal arrays (optional)
# np.savetxt('continuous_sine_wave.txt', continuous_sine_wave, delimiter=',')
# np.savetxt('discrete_sine_wave.txt', discrete_sine_wave, delimiter=',')
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.

The simulate_continuous_sine_wave function takes a time array, amplitude, frequency, and phase as input and returns an array representing the continuous sine wave signal. It calculates the sine wave signal using numpy.sin with the appropriate formula.

The simulate_discrete_sine_wave function takes the number of samples, sampling frequency, amplitude, frequency, and phase as input and returns an array representing the discrete sine wave signal. It first calculates the time array using the number of samples and sampling frequency and then calculates the sine wave signal using numpy.sin with the appropriate formula.

In the main part of the program, we define the time range for the continuous sine wave signal using numpy.linspace.

We define the number of samples, sampling frequency, amplitude, frequency, and phase for the discrete sine wave signal.

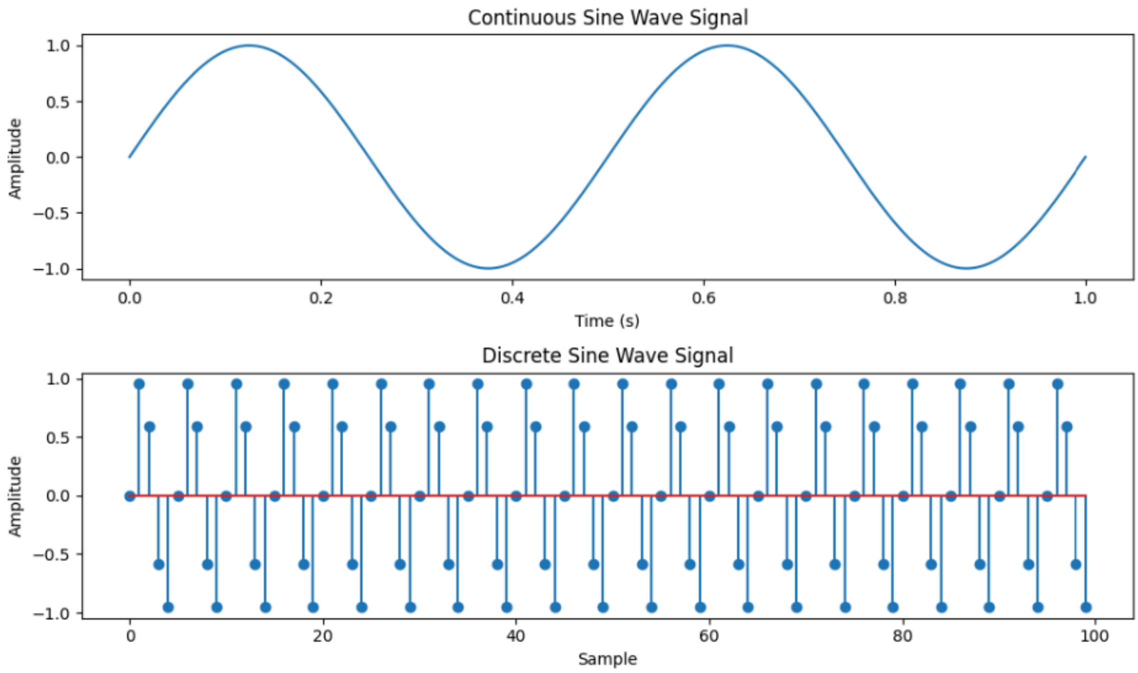
We simulate the continuous sine wave signal by calling the simulate_continuous_sine_wave function with the time array and parameters.

We simulate the discrete sine wave signal by calling the simulate_discrete_sine_wave function

with the number of samples, sampling frequency, amplitude, frequency, and phase.
We plot and display the continuous and discrete sine wave signals using matplotlib.pyplot.plot and matplotlib.pyplot.stem, respectively.
The sine wave signal arrays can be optionally saved to text files using numpy.savetxt.
You can modify the time range for the continuous sine wave signal, the number of samples, sampling frequency, amplitude, frequency, and phase for the discrete sine wave signal according to your requirements.
Uncomment the np.savetxt lines if you want to save the sine wave signal arrays to text files.
Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

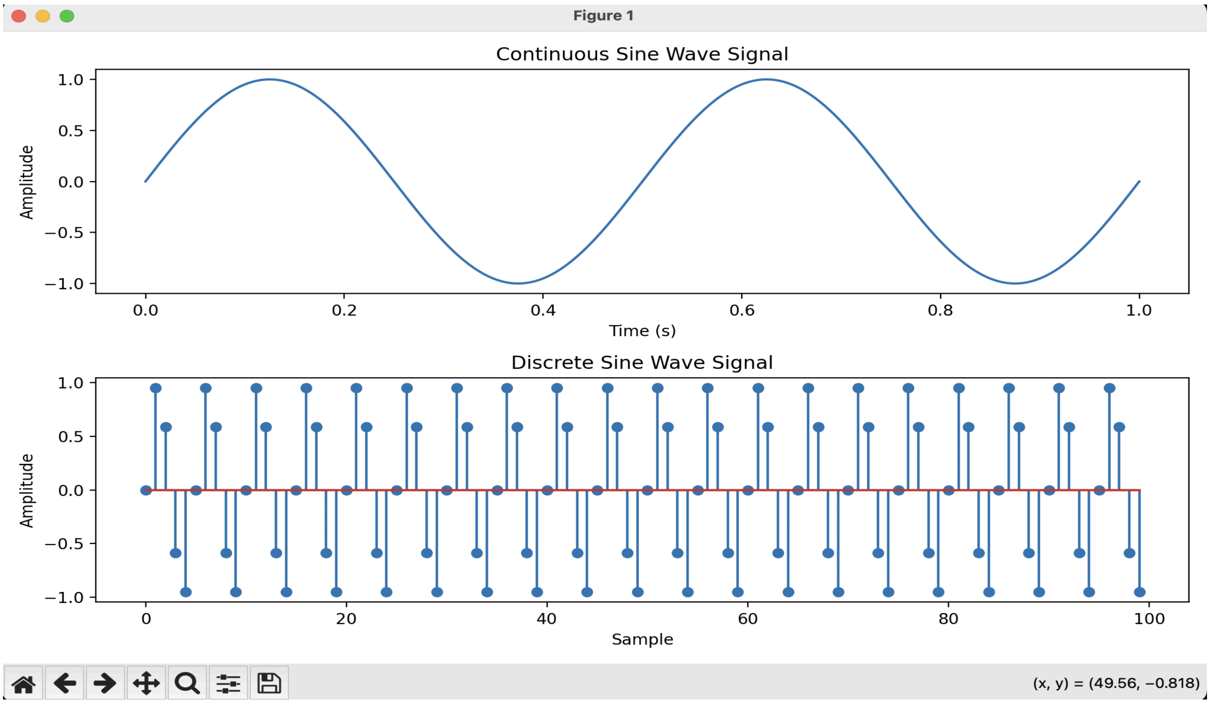
Expected Output:

Two plots will be displayed: the continuous sine wave signal and the discrete sine wave signal.
The sine wave signal arrays will be saved to text files (optional).
Feel free to experiment with different time ranges, numbers of samples, sampling




```python
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_sine_wave(time, amplitude, frequency, phase):
    sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
    return sine_wave
```

```
def simulate_discrete_sine_wave(num_samples, sampling_frequency, amplitude,
frequency,phase):

    time = np.arange(num_samples) / sampling_frequency
    sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
    return sine_wave

time = np.linspace(0, 1, 1000)

num_samples = 100
sampling_frequency = 10
amplitude = 1
frequency = 2
phase = 0
continuous_sine_wave = simulate_continuous_sine_wave(time, amplitude, frequency,
phase)

discrete_sine_wave = simulate_discrete_sine_wave(num_samples, sampling_frequency,
amplitude, frequency, phase)

plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_sine_wave)
plt.title('Continuous Sine Wave Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_sine_wave)
plt.title('Discrete Sine Wave Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```



AIM: Write a python program to simulate y(t)=u(t)+u(t-1)+3u(t+5).

Theory:

The unit step function, denoted as u(t), is a function that is 0 for negative time or indices and 1 for non-negative time or indices. It is commonly used to represent a sudden change or transition in a system.

The function y(t) = u(t) + u(t-1) + 3*u(t+5) is a combination of three unit step functions. It has a value of 1 at t=0, t=1, and t=-5, and a value of 0 elsewhere.

Flowchart:

The flowchart for the program will consist of the following steps:

Define the time range.

Create an empty array to store the function values.

Iterate over each time index.

Calculate the value of y(t) at each time index using the given function formula.

Plot and display the function values.

Program

Now, let's see the Python program that simulates y(t) = u(t) + u(t-1) + 3*u(t+5):

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
y = np.zeros_like(time)
y[time >= 0] = 1
```

```
y[time >= 1] += 1
y[time >= -5] += 3
return y
# Define the time range
time = np.linspace(-10, 10, 1000)
# Simulate the function
function_values = simulate_function(time)
# Plot and display the function
plt.plot(time, function_values)
plt.title('Function y(t) = u(t) + u(t-1) + 3*u(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 5.5])
plt.grid(True)
plt.show()
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.

The simulate_function function takes a time array as input and returns an array representing the function values. It creates an array of zeros with the same shape as the input time array and sets the values according to the function formula.

In the main part of the program, we define the time range using numpy.linspace.

We simulate the function by calling the simulate_function function with the time array.

We plot and display the function values using matplotlib.pyplot.plot.

We set the title, x-axis label, y-axis label, y-axis limits, and enable the grid.

Finally, we show the plot using matplotlib.pyplot.show.

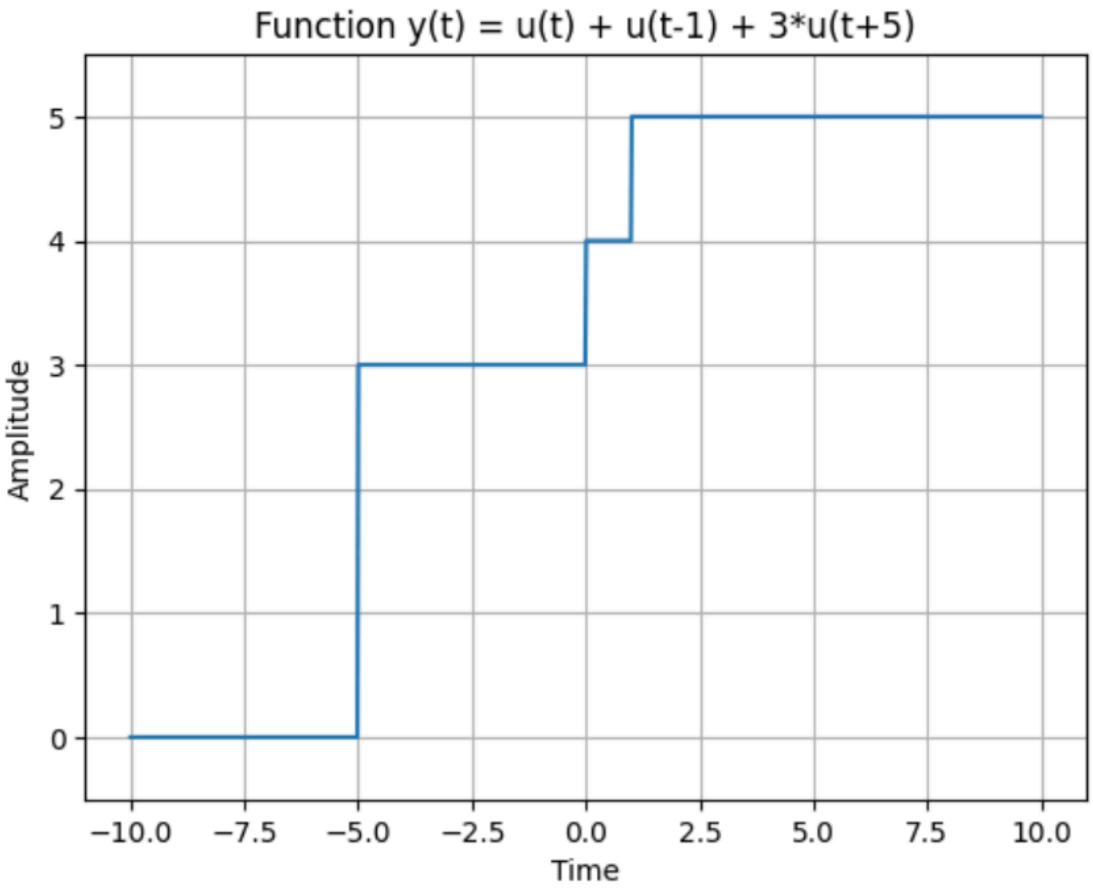
You can modify the time range according to your requirements.

Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

Expected Output:

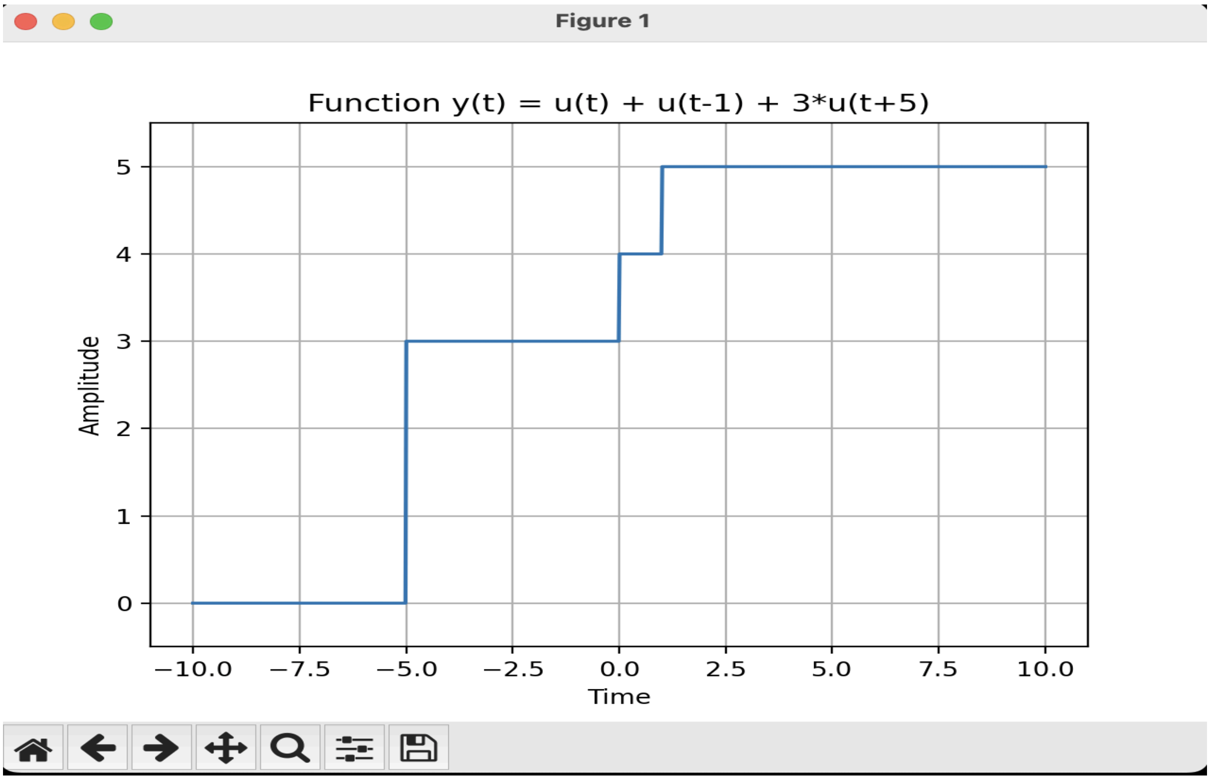
A plot will be displayed showing the function y(t) = u(t) + u(t-1) + 3*u(t+5).

The function will have a value of 1 at t=0, t=1, and t=-5, and a value of 0 elsewhere.




```
# y(t)=u(t)+u(t-1)+3u(t+5)

import numpy as np
import matplotlib.pyplot as plt

def simulate_function(time):
    y = np.zeros_like(time)
    y[time >= 0] = 1
    y[time >= 1] += 1
    y[time >= -5] += 3
    return y

time = np.linspace(-10, 10, 1000)

function_values = simulate_function(time)
```

```
plt.plot(time, function_values)
plt.title('Function y(t) = u(t) + u(t-1) + 3*u(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 5.5])
plt.grid(True)
plt.show()
```



AIM: Write a python program to simulate y(t)=Delta(t)+delta(t-1)+3*delta(t+5).

Theory:

The impulse function, denoted as Delta(t), is a function that is infinite at t=0 and zero elsewhere. It is often used to model instantaneous events.

The shifted impulse function, denoted as delta(t), is similar to the impulse function but is shifted by a certain amount. It is non-zero only at the shifted time.

The function y(t) = Delta(t) + delta(t-1) + 3*delta(t+5) is a combination of three impulse functions. It has an impulse at t=0, t=1, and t=-5, with different magnitudes.

Flowchart:

The flowchart for the program will consist of the following steps:

Define the time range.
Create an empty array to store the function values.
Iterate over each time index.
Calculate the value of y(t) at each time index using the given function formula.
Plot and display the function values.

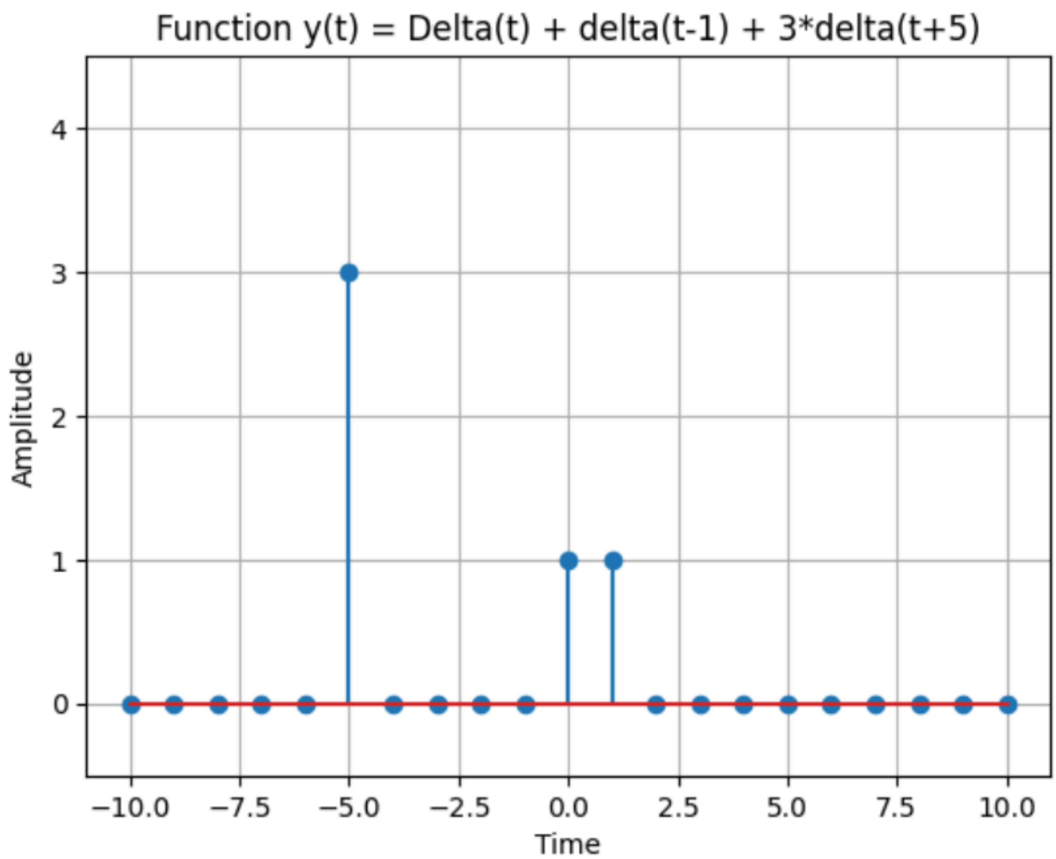
Now, let's see the Python program that simulates y(t) = Delta(t) + delta(t-1) + 3*delta(t+5):

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
y = np.zeros_like(time)
y[time == 0] = 1
y[time == 1] += 1
y[time == -5] += 3
return y
# Define the time range
time = np.arange(-10, 11)
# Simulate the function
function_values = simulate_function(time)
# Plot and display the function
plt.stem(time, function_values)
plt.title('Function y(t) = Delta(t) + delta(t-1) + 3*delta(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 4.5])
plt.grid(True)
plt.show()
```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.
The simulate_function function takes a time array as input and returns an array representing the function values. It creates an array of zeros with the same shape as the input time array and sets the values according to the function formula.
In the main part of the program, we define the time range using numpy.arange.
We simulate the function by calling the simulate_function function with the time array.
We plot and display the function values using matplotlib.pyplot.stem.
We set the title, x-axis label, y-axis label, y-axis limits, and enable the grid.
Finally, we show the plot using matplotlib.pyplot.show.
You can modify the time range according to your requirements.
Make sure you have numpy and matplotlib installed (pip install numpy matplotlib) before running this program.

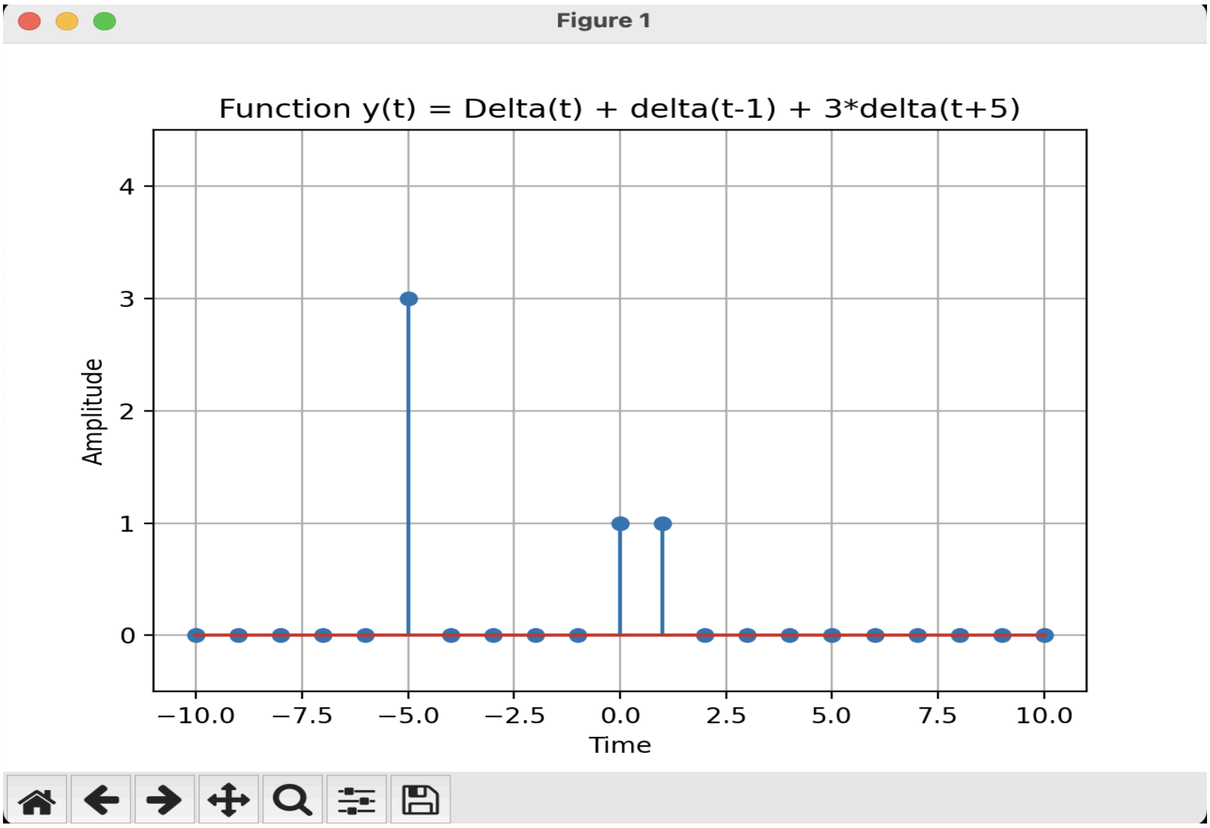
Expected Output:
A stem plot will be displayed showing the function y(t) = Delta(t) + delta(t-1) + 3*delta(t+5). The function will have impulses at t=0, t=1, and t=-5, with magnitudes of 1, 1, and 3 respectively, and zero elsewhere.




```
#y(t)=Delta(t)+delta(t-1)+3*delta(t+5)

import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time == 0] = 1
    y[time == 1] += 1
    y[time == -5] += 3
    return y

time = np.arange(-10, 11)
```

	Marwadi University Faculty of Engineering &Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Simulate discrete time sequences.	
Experiment No: 1	Date:19/07/2026	Enrollment No:92400133132

```
function_values = simulate_function(time)

plt.stem(time, function_values)
plt.title('Function y(t) = Delta(t) + delta(t-1) + 3*delta(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 4.5])
plt.grid(True)
plt.show()
```



Conclusions:

Signal Representation: Basic signals (impulse, step, ramp, exponential, parabolic, and sine wave) form the foundational building blocks of signal processing, each uniquely characterizing systems in both the continuous and discrete domains.

<table>
<tr><td rowspan="1"></td><td colspan="2">Marwadi University
Faculty of Engineering &Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Digital Signal and Image Processing (01CT1513)</td><td colspan="2">AIM: Simulate discrete time sequences.</td></tr>
<tr><td>Experiment No: 1</td><td>Date:19/07/2026</td><td>Enrollment No:92400133132</td></tr>
</table>

Mathematical Alignment: Python libraries like NumPy and Matplotlib provide a highly effective framework for accurately translating theoretical signal equations into verifiable dynamic simulations and graphical visualizations.

System Analysis: Simulating shifted composite signals (such as combinations of impulse or step functions) demonstrates how complex waveforms can be constructed, analyzed, and manipulated to model real-world system behavior and transitions.

GITHUB Link:-

https://github.com/BahopiyaDhaval/DSIP-Mu.git